

Building a POC that deploys to Cloudflare on day one

For small applications with a public front end and a private data source — a spreadsheet, a vendor API, a database. So a project is deployable from its first commit rather than retrofitted later, and the credential never reaches the browser.

The EO Ukraine calendar is the reference implementation. Every pattern here is running in it.

```

Browser  →  /api/<thing>  →  your server code  →  credential  →  private source
public    same origin    Cloudflare          secret        private

```

00

The one rule everything else serves

The browser is public. The server is not. Decide which side every value lives on before writing a line of code, and never move one across.

A key in front-end code is public no matter how it got there: a build-time variable inlined into a bundle, a `<meta>` tag, a “hidden” JSON file, a source map. If the browser can run it, someone can read it.

Two consequences that are easy to get wrong:

- **Everything your endpoint returns is public**, whether or not the UI displays it. Someone can open `/api/calendar` directly. Return the fields the interface renders, not the row you happened to fetch.
- **Error messages are part of the response.** A message naming the service account, quoting the upstream body, or echoing configuration leaks exactly when things are going wrong and people are pasting screenshots into chat.

01

Choose the platform, then stop thinking about it

Cloudflare offers two ways to host this. Both are current; neither is deprecated.

	PAGES + FUNCTIONS	WORKERS + STATIC ASSETS
Static files	build output directory	<code>assets.directory</code>
Server code	<code>functions/</code> file routing	one entry, your own routing
Set up by	connecting a Git repo	<code>wrangler deploy</code> , or Git
SSR frameworks	limited	the supported target
Cron · Queues · Durable Objects · Email	no / limited	yes
Best for	static or SPA plus a few endpoints	a framework server, or scheduled work

Default to Pages + Functions for a POC of this shape: a front end plus a few read endpoints. Least machinery, file routing needs no code, a Git connection deploys it.

Choose Workers + static assets when any of these is true — and decide at the start rather than migrating later:

- the front end is Nuxt, SvelteKit, Next or Astro SSR and you want server rendering
- you need a **cron trigger**, a **queue consumer**, **Durable Objects**, or **email**
- you want `run_worker_first` — the Worker seeing a request before assets do

```
// wrangler.jsonc
{
  "name": "my-app",
  "main": "src/server.js",
  "compatibility_date": "2026-09-02",
  "assets": {
    "directory": "./dist",
    "binding": "ASSETS",
    "not_found_handling": "single-page-application"
  }
}
```

Routing there: an exact asset match is served without running the Worker; anything else runs the Worker, which can fall back with `env.ASSETS.fetch(request)`.

Framework projects use the framework's Cloudflare adapter — Nitro preset `cloudflare_module` for Nuxt, `@sveltejs/adaptor-cloudflare`, `@opennextjs/cloudflare`. Everything in this document still applies: same variable names, same server-only boundary, same local parity, same degradation. Only the file layout differs, because the adapter decides it.

Repository layout

The layout is the security boundary made visible. If a reviewer has to think about whether a file is published, the layout is wrong.

<code>public/</code>	← THE ONLY DIRECTORY PUBLISHED. Nothing secret, ever.
<code>index.html</code>	
<code>assets/</code>	
<code>src/</code>	front-end modules
<code>data/</code>	demo fallback data
<code>_headers</code>	security headers and cache policy
<code>functions/</code>	← Pages Functions. REPOSITORY ROOT, NOT inside <code>public/</code>
<code>api/calendar.js</code>	→ <code>/api/calendar</code>
<code>lib/</code>	server-only: auth, parsing, the endpoint's answer
<code>data/</code>	inputs that generate the demo data
<code>docs/</code> <code>tools/</code>	specifications, tests, dev helpers
<code>serve.mjs</code>	local dev server. Never deployed
<code>.env</code>	gitignored

Three placements that are wrong in ways that are not obvious:

1. **`functions/` must be at the repository root, not beside `index.html`** . Put it inside the build output and Cloudflare publishes your server code as a readable static file and never executes it. If the front end lives in `public/` , `functions/` is its *sibling*.
2. **`lib/` must be outside the output directory**, or the module that builds a JWT ships to the browser.
3. **The output directory contains only what should be public**. Pointing Cloudflare at the repository root publishes your docs, your READMEs and your generator scripts.

Verify it rather than believing it:

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:PORT/lib/google-sheets.mjs    # 404
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:PORT/.env                  # 404
```

Configuration and secrets

The same variable names in every environment. A name that differs between local and production is a bug that only appears in production.

WHERE	HOW
Cloudflare	Settings → Variables and secrets. Secrets for credentials, plain variables for flags
Local	a gitignored <code>.env</code> — or <code>.dev.vars</code> if you standardise on Wrangler. Pick one
Pages Function	<code>context.env.NAME</code>
Worker	<code>env.NAME</code>
Local server	<code>process.env</code> / your <code>.env</code> loader

process.env does not exist in the Workers runtime. Server code that must run in both should take an env object as an argument rather than reaching for a global.

Commit a `.env.example` with names and empty values. Never a real one.

```
.env
.env.*
!.env.example
.dev.vars
.dev.vars.*
.wrangler/      # miniflare caches request and response bodies on disk
node_modules/
```

CREDENTIAL FILES

A service-account JSON or downloaded key is best kept **outside the repository entirely**, with local config pointing at wherever it was downloaded. If a project insists on holding one, it goes in a directory ignored *wholesale*:

```
secrets/*
!secrets/README.md    # the directory documents itself; nothing else is committable
```

Add the ignore rule **before** copying the file in, so it is never visible to git for even one command. Then prove it: `git check-ignore -v <path>` and `git add -An | grep secrets`.

A PRIVATE KEY MUST NEVER HAVE BEEN COMMITTED

Check history, not just the working tree — and check for *values*, not variable names, which legitimately appear in source:

```
git rev-list --all --objects | awk '{print $1}' | sort -u | while read -r o; do
  git cat-file -t "$o" 2>/dev/null | grep -q blob && git cat-file -p "$o" 2>/dev/null
done | grep -cE -- "-----BEGIN [A-Z ]*PRIVATE KEY-----|\"private_key_id\""
```

Anything but **0** means the key must be **revoked and reissued**. Deleting it in a later commit does not help; the history is already pushed.

04

One endpoint contract, two implementations

The front end must not know which environment it is in. Give it one URL and change what answers it.

```
public/src/api/http.js → GET /api/calendar
                        |— production: functions/api/calendar.js (context.env)
                        |— local:      serve.mjs                (.env)
                        both call lib/
```

Neither adapter contains logic. They read the environment, call the shared module, serialise the result, and log. That is what stops the two environments drifting — the only difference between them is where the secret came from.

Do not branch the front end on an environment flag. A flag like `CLOUDFLARE_ON` is fine as server-side configuration for a log line, but the moment browser code reads it you have two front ends to test and a name that hints at your infrastructure. If the endpoint answers the same everywhere, the flag is unnecessary in the browser by construction.

Write server code to the Web Platform, not to Node. `fetch`, `crypto.subtle`, `Request / Response`, `URL`, `TextEncoder` exist in both the Workers runtime and modern Node. A shared module using only those runs in both with no compatibility flag. Reach for a Node built-in and you have just split your codebase in two.

05

The server module, in four parts

Keep them separate. Each is independently testable, and the boundaries are where the mistakes live.

SERVER `LIB/<SOURCE>.MJS — AUTHENTICATE AND FETCH`

Nothing else. It returns raw upstream data and throws typed errors.

```
export class SourceError extends Error {
  constructor(reason, message, { status, publicMessage } = {}) {
    super(message);          // SERVER LOG: may name the account, quote upstream
    this.reason = reason;    // short machine-readable tag; safe to publish
    this.publicMessage = publicMessage || 'The data could not be read.'; // BROWSER
  }
}
```

Two messages is the whole trick. One message inevitably gets returned to the client with the upstream's error payload inside it.

SERVER LIB/NORMALIZE.MJS — RAW DATA INTO THE RENDER SHAPE

Every quirk of the source is handled here, once, and *recorded* rather than silently patched:

```
issues.push({ row, kind: 'unreadable_date', detail: `"${raw}" is not YYYY-MM-DD` });
```

A repair hidden inside a template is invisible the next time the source changes. A repair recorded as data shows up in a log and can be fixed upstream.

It is also the boundary internal fields must not cross. Name them and strip them explicitly:

```
const INTERNAL = new Set(['notes', 'owner', 'internal_comments']);
export const stripInternal = (r) =>
  Object.fromEntries(Object.entries(r).filter(([k]) => !INTERNAL.has(k)));
```

SERVER LIB/<DOMAIN>.MJS — WHAT THE ENDPOINT ANSWERS

Takes the environment and the query, returns `{ body, logLine, issues }`. The adapters print `logLine` and serialise `body`. This split is what keeps a service-account address out of a JSON response while still putting it in a log where an operator needs it.

IDENTIFIERS DESERVE THEIR OWN PARAGRAPH

Never derive an id from row position or encounter order. A “first one wins, later ones get -2” scheme means re-sorting the source swaps two records' URLs — and a shared link then opens **the wrong thing**, which is worse than opening nothing, because nobody notices.

Compose the id from values a person actually chose (`title-slug` + `date`). Assign ids in **one pass over the whole set**, so a collision suffixes *every* member of the group rather than just the later one — otherwise the first row met still keeps the clean id and you have reinvented the bug. Offer an optional explicit `Slug` column for records needing a permanent address.

06

Degrade, do not crash

A POC is demonstrated more often than it is used. It must never show a stack trace to a room.

```
{ ok: true, source: 'live', count, fetched_at, events: [...] }
{ ok: false, source: 'mock', reason: 'not_shared', message: '<generic>' }
```

- **HTTP stays 200 for a handled failure.** A 500 puts a red line in the console for something the application handled correctly, and teaches whoever is on call to ignore red lines.
- **The client falls back to demo data** committed under the output directory. This also covers the case where no server code is deployed at all.

- **Say so in the interface.** A quiet banner naming the reason. Silently showing last season's data is how a broken integration survives for weeks.
- **Distinguish a chosen demo from a failure.** A `DEMO=1` flag that pins the app to demo data and contacts nothing is worth having — for a rehearsal, or while the source is being restructured. Give it different wording, or a deliberate choice looks like a fault and a fault looks intentional.
- **An empty source is a failure, not an empty result.** Zero rows is a misconfiguration far more often than a genuinely empty dataset, and rendering nothing hides it.

07

Fetch what a screen needs, when it needs it

Measure before optimising. The instinct is that a slow response means too much data. Often it is round-trip latency, which chunking multiplies.

Measured on the reference project:

sign JWT + exchange token		187 ms
read ALL 22 columns		476 ms
read 1 column		422 ms

One column costs the same as twenty-two. Splitting that read into three chunks would have cost **~1.6s instead of ~0.7s**. Worse, spreadsheet rows are in the order a human typed them, not date order — so “the next ten records” cannot be a row range at all. Everything must be read and sorted before the first card is even correct.

So put the laziness everywhere except there:

- **Cache the access token** until it nears expiry. Signing a fresh one per request throws away a third of a cold request, every request. Module scope is enough — both runtimes reuse it between requests, and a cold isolate simply mints one.
- **Cache the parsed source briefly** (30–120s) so a detail request straight after a list request costs nothing upstream. Keep it short: someone editing the source expects to see it.
- **Split the payload by need, not by size.** A list endpoint returns what a card draws; a detail endpoint returns the long prose for one record. Withhold on the server, not in the UI.
- **Fetch a scope only when asked.** “Past events” is a second request triggered by the button, not a filter over one big payload.
- **Cache each result in the client until reload**, keyed by scope and by id — the granularity the screens ask in.
- **Paginate the DOM, not the network.** Render a page of cards and append on an `IntersectionObserver` sentinel. Two hundred cards with photographs is layout work nobody scrolling past the third one asked for.
- **Paint from what you already have.** A detail page can render its header from the list record instantly and fill in prose when it lands, instead of a spinner over facts already in memory.

```
// a success caches at the edge; a failure never does, so recovery is immediate
const cacheControl = body.ok
  ? 'public, max-age=0, s-maxage=120, stale-while-revalidate=600'
  : 'no-store';
```

Local development must be the real thing

Two rules, and they are the difference between an hour of debugging and a week of it.

One command, real data. `npm install && npm run dev` should show live data from the real source, using local credentials. A local mode that only shows fixtures means nobody exercises the integration until deployment.

Parity enforced by sharing files, not by discipline. The local server applies the same headers file the platform will. That single choice caught three real defects in the reference project before they could ship: an inline `<script>` violating `script-src`, three inline `style` attributes, and a wrong relative path that would have left the deployed calendar empty.

Keep a second command that runs the actual platform runtime, so the deployed code path is exercised before it is deployed:

```
{
  "scripts": {
    "dev": "node serve.mjs",
    "dev:cf": "node tools/dev-cf.mjs",          // wrangler pages dev <output-dir>
    "test": "node tools/test-parser.mjs && node tools/test-api.mjs"
  }
}
```

Have `dev:cf` read the project's single `.env` and pass values as `--binding` arguments, rather than maintaining a second secrets file that drifts.

Security headers

Ship `_headers` in the output directory (Pages), or set them in the Worker response.

```
/*
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'
  'unsafe-inline' https://fonts.googleapis.com; font-src https://fonts.gstatic.com;
img-src 'self' data: https; connect-src 'self'; frame-ancestors 'self';
base-uri 'none'; form-action 'none'
```

Keep `script-src 'self' strict` — no `'unsafe-inline'`. It is the directive that stops an injected `<script>` running, and it matters most in exactly the applications this standard describes, where content is authored in a shared document. It costs one thing: no inline `<script>` and no inline event handlers, both easy to avoid.

`style-src` may need `'unsafe-inline'` if you sanitise author HTML — merely *parsing* a `style` attribute raises a CSP report even when it is stripped before rendering. Relax that one and say why in a comment; do not relax `script-src` to match.

If you render HTML written by a human in the source document, sanitise it from an allowlist.

Rebuild the tree keeping known tags, unwrap the rest to their text, drop every attribute except a safe href, and force target="_blank" rel="noopener noreferrer". Treat pasted word-processor markup as the normal case, not the exception.

10

Prove it, do not claim it

Tests should assert the things a reviewer would otherwise have to take on faith.

A source parser test, with fixtures and no network: field mapping, malformed input degrading rather than throwing, unsafe URLs dropped, internal fields absent from output, and — if ids are derived — that reordering the input produces identical ids.

An endpoint test that stubs the upstream and asserts the *shape* and the *call count*:

```
let calls = [];  
globalThis.fetch = async (url) => { calls.push(url); /* return a fixture */ };  
  
await getCalendar(env, { scope: 'upcoming' });  
check('a warm request re-reads nothing', calls.length === 0);
```

Counting requests is what turns “we made it lazy” from a claim into a fact.

A leak scan in a real browser. Load every page, collect every response body, and assert the credentials are absent:

```
[['private key', /-----BEGIN [A-Z ]*PRIVATE KEY-----/],  
 ['token',      /\bya29\.[A-Za-z0-9_-]{20}/],  
 ['account',    /iam\.gserviceaccount\.com/],  
 ['source id',  /spreadsheets\/d\/|SPREADSHEET_ID/]]
```

Screenshots that fail on a console error. A script that renders every view at desktop, tablet and phone widths and exits non-zero on any console error catches the class of breakage unit tests never see.

11

Order of work

Each of these is cheap now and expensive later.

- 1 `git init`, and write `.gitignore` **before** any credential exists on disk.
- 2 Decide Pages or Workers using the table in §01. Create the project, connect the repo.
- 3 Create the output directory with a placeholder page. Deploy. **Get a green deployment before there is anything to debug.**
- 4 Add `functions/api/health.js` returning `{ ok: true }`. Deploy. Now routing is proven.

- 5 Set the environment variables in the dashboard, and the same names in `.env`.
- 6 Build `lib/` — auth, then normalise, then the endpoint's answer. Test with the upstream stubbed.
- 7 Wire the local server to the same `lib/`. Confirm real data locally.
- 8 Build the front end against the endpoint, with the demo fallback from the first commit.
- 9 Add `_headers`, and make the local server apply them. Fix what that breaks.
- 10 Write the README: architecture, both setups, variable **names**, commands, and the route.

12

Checklist before the first deploy

- ☐ Output directory contains only public files; `curl` proves `lib/`, `.env`, `docs/` are 404
- ☐ `functions/` at the repository root, not inside the output directory
- ☐ Same variable names locally and in the dashboard; `.env.example` committed with no values
- ☐ `.env`, `.dev.vars`, `.wrangler/`, `node_modules/`, credential files all gitignored
- ☐ Git history contains no private key **value** — checked with the blob scan
- ☐ Server code uses `context.env` / `env`, never `process.env`
- ☐ Server code uses only Web Platform APIs, or a compatibility flag is set deliberately
- ☐ Endpoint returns only rendered fields; internal columns stripped at a named boundary
- ☐ Error responses are generic; detail goes to the log only
- ☐ Ids are not derived from row position or encounter order
- ☐ Failure falls back to demo data, says so in the UI, and returns 200
- ☐ Token cached; successful responses cached at the edge; failures never cached
- ☐ `_headers` applied locally as well as in production
- ☐ `npm install && npm run dev` shows real data
- ☐ `npm test` passes; screenshots render with no console errors
- ☐ README documents architecture, both setups, variable names, commands, and the route

Cloudflare POC standard · v1 · reference implementation: `eo-ukraine-calendar-app` · the same text lives at `docs/CLOUDFLARE-POC-STANDARD.md`